Clase 2

Introducción al Deep Learning y el Procesamiento del Lenguaje Natural

Inteligencia Artificial para Ingeniería Biomédica

Paulo Veiga & Marco Sánchez Sorondo
Universidad Austral — Facultad de Ingeniería

Agenda

Objetivo: Entender cómo se puede generar texto automáticamente.

1. Repaso: problemas clásicos de ML
2. NLP: motivación y problemas
3. Language Modelling: predecir la siguiente palabra
4. Representación de texto: tokens, vocabulario y embeddings
5. Redes neuronales: del perceptrón al deep learning
6. De RNNs a Transformers: el rol de attention
7. Cómo funciona un LLM
8. Training vs. Inference

Repaso: La Taxonomía de Problemas de IA

¿Qué tipos de problemas resuelve la IA?

Predicción

Aprender X → Y a partir de datos etiquetados. Incluye clasificación y regresión.

Percepción

Extraer estructura de señales sensoriales: imagen, audio, video.

Representación

Aprender embeddings y espacios latentes que capturan relaciones entre datos.

Decisión Secuencial

Maximizar recompensa acumulada. Formalizado como Reinforcement Learning.

Búsqueda / Planificación

Encontrar la mejor secuencia de acciones en un espacio de estados.

Razonamiento Simbólico

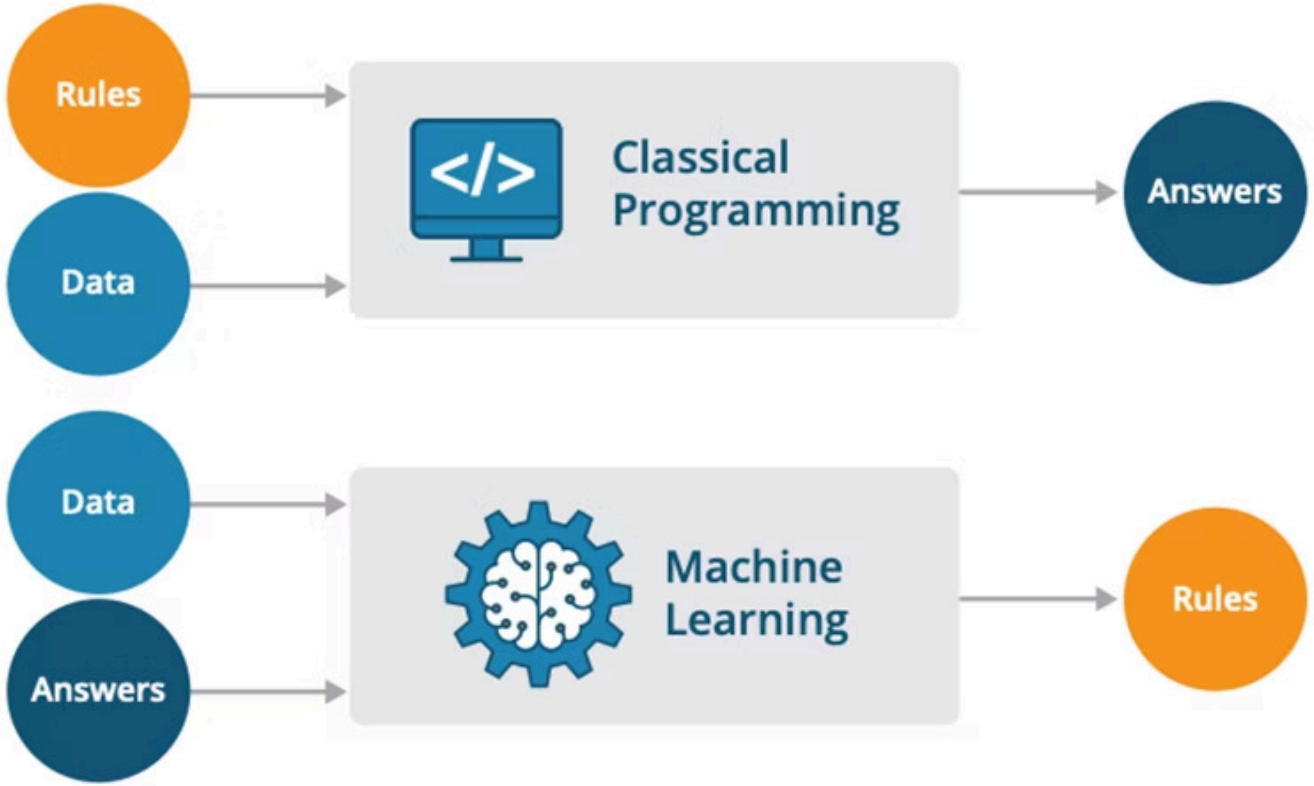
Manipular símbolos y reglas IF–THEN para derivar conclusiones lógicas.

Generación

Producir nuevas muestras coherentes: texto, imagen, audio, código. Infinita cantidad de outputs.

💡 Un auto autónomo combina percepción + decisión secuencial + planificación

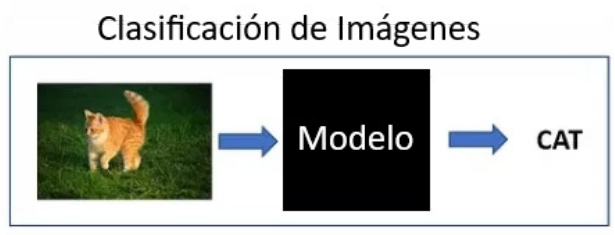
Machine Learning: Cómo funciona la IA moderna

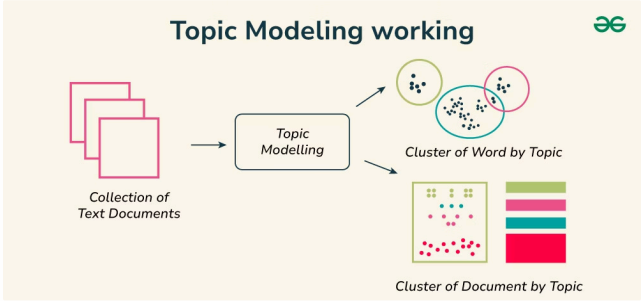


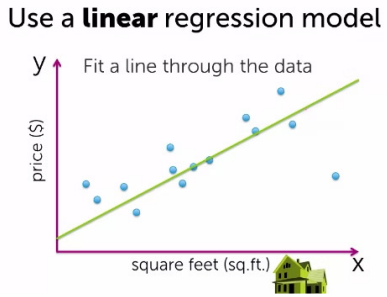
Problemas clásicos de ML

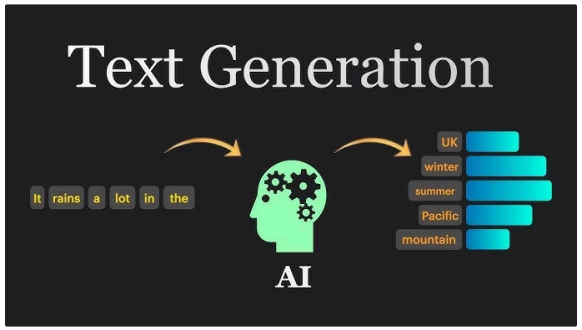
¿Como se formulan concretamente los problemas vistos?

- Clasificación (Prediccion) → ¿A qué categoría pertenece? (ej: ¿este email es spam o no?)
- Regresión (Prediccion) → ¿Qué valor va a tener? (ej: ¿cuánto va a costar esta casa?)
- Clustering (Representación) → ¿Qué datos se parecen entre sí? (ej: agrupar pacientes con síntomas similares)
- **Generación → ¿Puedo crear datos nuevos? (ej: generar imágenes, texto)**









El modelo aprende patrones de los datos, no se programa con reglas explícitas.

Problemas clásicos de ML aplicados a texto

Cada problema clásico de ML tiene un equivalente cuando el input es texto.

Problema ML	¿Qué resuelve?	Ejemplo con texto
Clasificación	Asignar una categoría	¿Este review es positivo o negativo?
Clasif. multiclase	Elegir entre varias categorías	¿En qué idioma está escrito?
Clustering	Agrupar datos similares	Agrupar noticias por tema
Predicción	Completar información	The ___ is on the mat → cat
Generación	Crear datos nuevos	The cat is on the ___ → mat

Sentimiento: "Qué bien comimos en el restaurante" → positivo
"No vuelvo nunca más a comer" → negativo

```
input (texto) → modelo → output
```

Todas requieren alguna manera de comprender el texto. ¿Cómo hacemos para que un modelo entienda texto?

¿Por qué procesar texto automáticamente?

Cómo las máquinas pueden analizar grandes volúmenes de texto.

Vivimos rodeados de texto: emails, historias clínicas, papers, redes sociales, reportes. Hay demasiado texto para que un humano lo procese manualmente.

- Analizar opiniones de miles de pacientes sobre un tratamiento
- Clasificar papers científicos por área
- Detectar automáticamente el idioma de un documento
- Responder preguntas a partir de un texto largo
- Resumir un paper de 30 páginas en un párrafo

El campo que se ocupa de esto se llama Procesamiento del Lenguaje Natural (NLP).

Problemas de NLP

Clasificación	Clasif. Multiclase	Clustering	Predicción	Generación
Detección de spam (spam / no spam)	Detección de idioma (español, inglés, francés...)	Agrupar noticias por tema	Autocompletar palabras en un buscador	Resumen automático de textos
Análisis de sentimiento (positivo / negativo)	Clasificación de intención del usuario (comprar, preguntar, quejarse...)	Agrupar reseñas similares de productos	Predecir la siguiente palabra en una oración	Traducción automática
Detección de discurso de odio (sí / no)	Etiquetado gramatical (sustantivo, verbo, adjetivo...)	Segmentar clientes por estilo de escritura	Completar huecos en texto (masked LM)	Generación de respuestas en chatbots
Detección de noticias falsas (real / falsa)	Clasificación de emociones (alegría, tristeza, enojo, miedo...)	Descubrir tópicos en un corpus (topic modeling)	Predecir la puntuación de una reseña	Generación de código a partir de texto
Detección de paráfrasis (sí / no)	Clasificación de tickets de soporte (facturación, técnico, ventas...)	Agrupar documentos legales por área del derecho	Predecir palabras faltantes en OCR dañado	Parafraseo automático de oraciones

¿Cómo se resolvían estos problemas?

Enfoque tradicional

El trabajo del data scientist consistía en:

01

Recolectar datos (conseguir textos relevantes)

02

Etiquetar datos (un humano marca: "esto es positivo", "esto es negativo")

03

Entrenar un modelo específico para cada tarea

Un modelo para sentimiento, otro para traducción, otro para NER...
Cada tarea requería su propio pipeline, sus propios datos etiquetados, su propio modelo.
Era costoso y lento.

Modelos de Generación de Palabras

Problema de Language Modelling.

Dado:

- un vocabulario V
- un corpus de texto T
- una frase incompleta P

El objetivo es predecir cuál es la siguiente palabra más probable.

Importante

El modelo no devuelve directamente una palabra, sino un vector de probabilidades sobre todo el vocabulario.

- una posición por cada palabra del vocabulario
- cada valor indica la probabilidad de que esa palabra sea la siguiente

La longitud del vector es |V|.

Modelos de Generación de Palabras – Ejemplo

```
V = { "a", "to", "on", "the", "sat", "cat", "dog", "bed", "ran", "mat", " ", "<fin>", ".", "," }
```

```
T = {
  "the cat sat on mat"
  "a dog ran to bed"
  "the dog sat on bed"
  "a cat ran to mat"
  "the cat ran to dog"
}
```

P =

```
the dog sat on
```

En el corpus, la palabra correcta corresponde a T[2] = "bed".
El vector objetivo tendrá un 1 en la posición correspondiente a "bed" y 0 en el resto.

Salida del modelo (vector de probabilidades sobre el vocabulario):

```
[a: 0.01, to: 0.02, on: 0.03, the: 0.04, sat: 0.01, cat: 0.01, dog: 0.02,
 bed: 0.78, ran: 0.03, mat: 0.04, " ": 0.01, <fin>: 0.00, ".": 0.00, ",": 0.00]
```

Sidetrack: Aprendizaje (Learning)

Cómo los modelos aprenden a generar texto (y cómo aprenden a resolver todas las tareas que vimos) a partir de los datos (corpus T)?

Concepto: Entrenamiento

¿Cómo aprende un modelo a predecir la siguiente palabra a partir del corpus?

0. El modelo por dentro tiene "clavijas" (parámetros) internas que hay que ajustar para que predigan correctamente la siguiente palabra mas probable.

01

Input: el modelo recibe una frase incompleta → "El gato está en la"

02

Predicción: genera una distribución de probabilidades

03

Comparación: la palabra mas probable era "alfombra".

04

Error (Loss): se calcula qué tan lejos estuvo de la palabra mas probable.

05

Ajuste: se modifican los parámetros para que el error sea menor

06

Repetición: se hace millones de veces con millones de frases

07

Resultado: un modelo que sabe "hablar".

Nos quedamos con los parámetros que minimizan el error.

Conceptos clave

- **Loss (pérdida):** número que mide qué tan equivocado estuvo el modelo
- **Optimización:** proceso de ajustar los parámetros para reducir la loss

¿Qué le vamos a enseñar a nuestro modelo?

En nuestro caso: a generar palabras. Pero el mismo método sirve para resolver cualquiera de los problemas ya vistos.
El primer paso para generar palabras (o para cualquier tarea de NLP), es lograr que el modelo **entienda qué significa.** Para eso, necesitamos representarlas como números (vectores).

Representación de palabras: Tokens

Los modelos trabajan con números, no con letras. El primer paso es dividir el texto en tokens — las unidades mínimas que el modelo procesa. Cada token tendrá un vector asignado que lo identifica.

Un token puede ser: una palabra completa, parte de una palabra, o un signo de puntuación.

```
"the cat sat on the mat"
→ ["the", "cat", "sat", "on", "the", "mat"]    (6 tokens)
```

Palabras vs Tokens

Los modelos modernos usan sub-palabras (o incluso caracteres):

```
"unbelievable"
→ ["un", "believ", "able"]    (3 tokens)
```

- Vocabularios más compactos (no necesita una entrada por cada palabra del idioma)
- Entiende palabras nuevas descomponiéndolas en partes conocidas
- Funciona en múltiples idiomas

Vocabulario y Representaciones de Texto

El vocabulario V es el conjunto completo de tokens que el modelo conoce. Por eso predice un vector de probabilidades de tamaño |V|. Pero el token (texto) en sí no puede ser procesado con los parámetros del modelo (números) directamente. Hay que convertir los tokens en números…

Formas de representar texto numéricamente

Bag of Words / TF-IDF	Contar palabras. Ignora orden y significado.
One-Hot Encoding	Vector con un solo 1. No captura similitud ("cat" y "dog" igual de lejos que "cat" y "refrigerator"). Cada vector canónico se corresponde con un token.
Embeddings	Vectores donde tokens con significado similar tienen vectores similares. Es lo que usan los modelos modernos.

Word2Vec

Uno de los primeros métodos para aprender embeddings automáticamente.

Idea: "una palabra se define por la compañía que tiene".

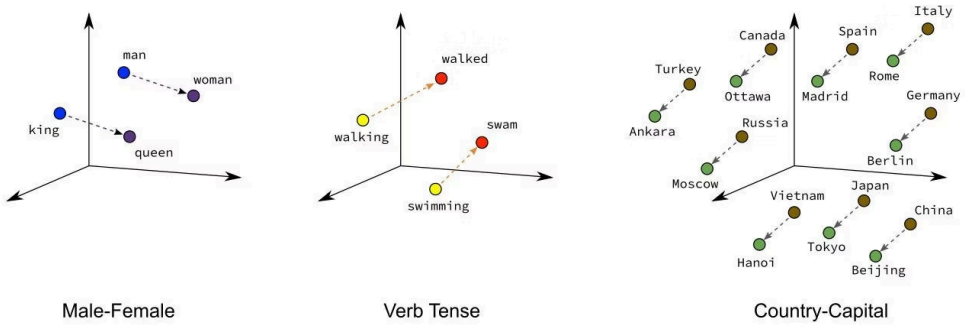
Funcionamiento: Saca promedios de los vectores de las palabras que circundan a cada una en el corpus de texto en que aparece.

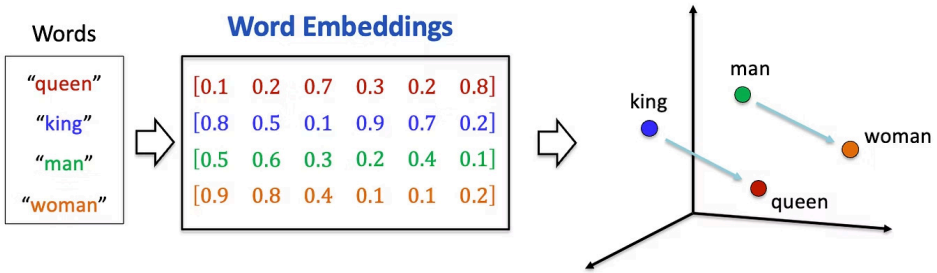
Importante: Word2Vec **no es un modelo de lenguaje (no genera palabras)**. Es un método para obtener embeddings con significado real de palabras en su contexto.

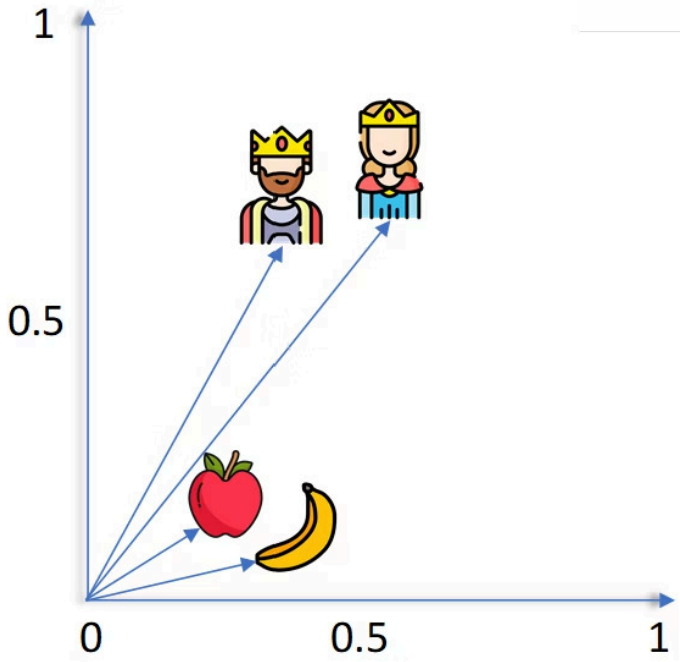
Embedding de texto: vector que codifica el significado de una palabra.

Palabras relacionadas y con significados similares se encuentran "cercanos" en el espacio.

También codifican relaciones entre palabras: el vector que me lleva de "man" a "woman" es el mismo que me lleva de "king" a "queen"....





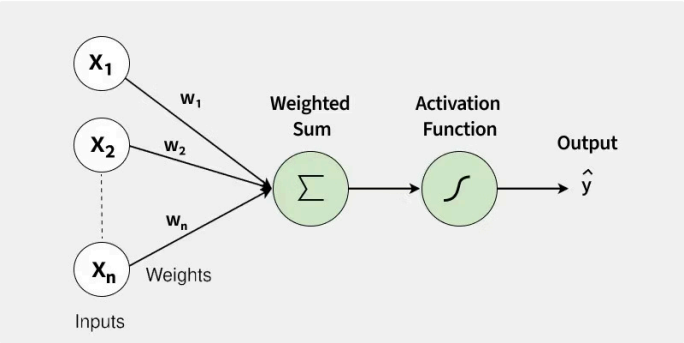


Redes Neuronales: Perceptrón

El bloque fundamental de las redes neuronales — la neurona artificial más simple.

¿Cómo funciona?

1. Recibe varios inputs (números)
2. Cada input se multiplica por un peso (importancia de ese input)
3. Se suman todos los productos + un bias (valor de ajuste)
4. El resultado pasa por una función de activación → output



$$\text{output} = \text{activación}(\, x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 + \text{bias} \,)$$

Los pesos y el bias son los parámetros que el modelo aprende durante el entrenamiento.

Analogía: es como tomar una decisión. Tenés varios factores (inputs), cada uno te importa más o menos (pesos), y al final tomás una decisión (output). El bias es tu predisposición inicial.

En NLP:

x_1,...,x_n es la representación numérica de una palabra (o de una frase). ¿Cómo sacamos el x_1,...,x_n de una frase? Word2Vec saca el promedio (obs: se pierde información del orden).

Redes Neuronales Profundas (Deep Learning)

¿Qué pasa si apilamos muchas capas de perceptrones?
Obtenemos una red neuronal profunda — de ahí viene el nombre Deep Learning.

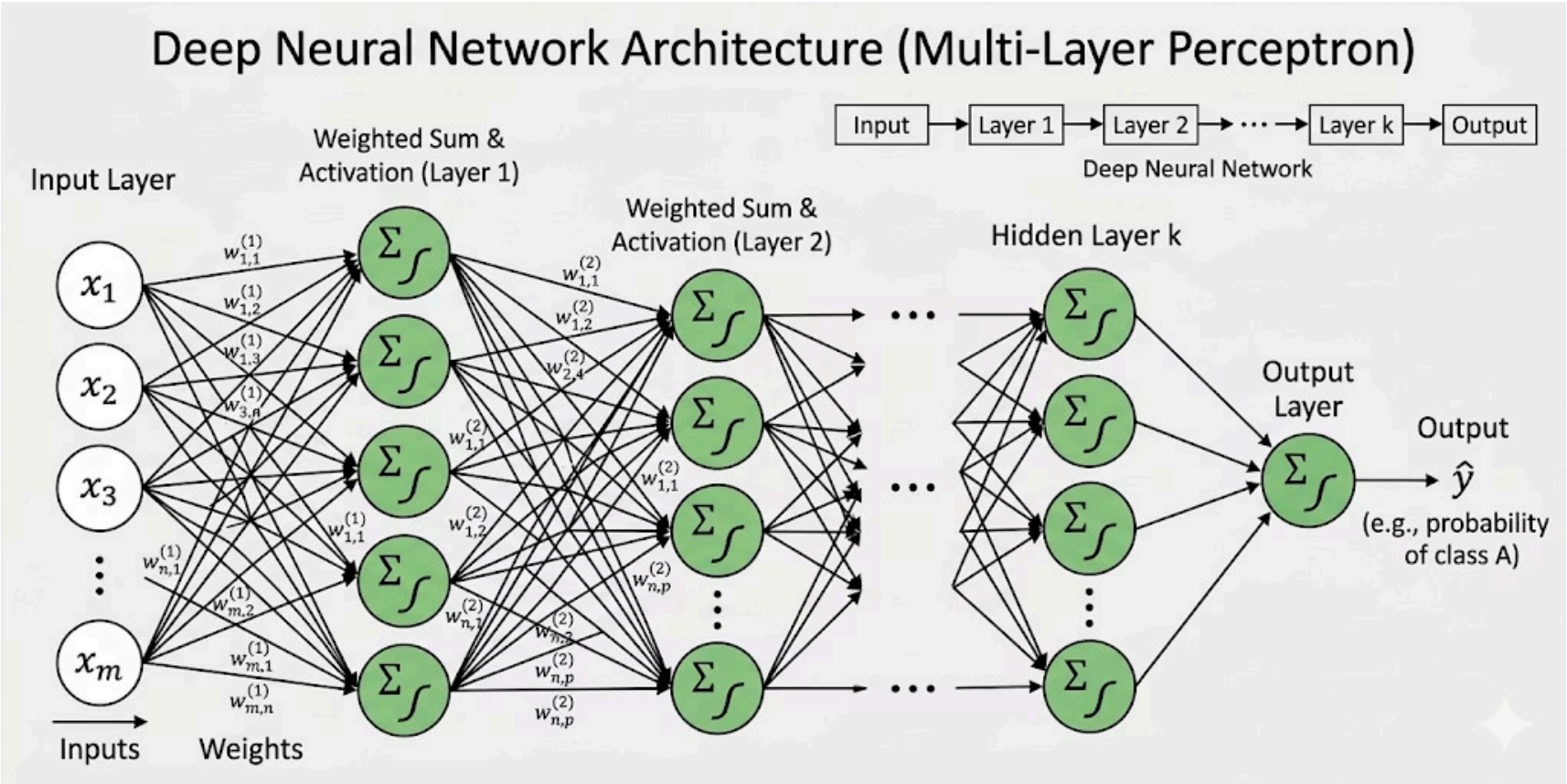
Cada capa transforma la información en una representación más abstracta:

- **Capas iniciales:** detectan patrones simples (combinaciones de letras, formas básicas)
- **Capas intermedias:** detectan patrones más complejos (palabras, frases, relaciones)
- **Capas finales:** capturan conceptos de alto nivel (significado, intención, contexto)

Más capas = más parámetros = mayor capacidad de modelar relaciones complejas.
Pero también = más datos necesarios para entrenar y más cómputo. Es un trade-off.

Los modelos de lenguaje modernos tienen miles de millones de parámetros (GPT-4, Claude, LLaMA).

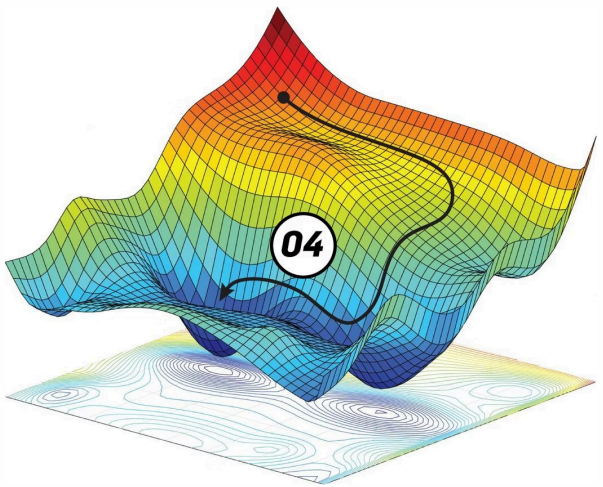
Deep Neural Network Architecture (Multi-Layer Perceptron)



Entrenamiento de Redes Neuronales: descenso de gradiente

¿Cómo aprenden los pesos correctos? El proceso es iterativo:

1. Se le da un ejemplo al modelo
2. El modelo genera una predicción con sus pesos actuales
3. Se compara la predicción con la respuesta correcta
4. Se calcula la loss (qué tan lejos estuvo)
5. Se ajustan los pesos un poquito en la dirección que más reduce la loss (-∇)
6. Se repite con el siguiente ejemplo

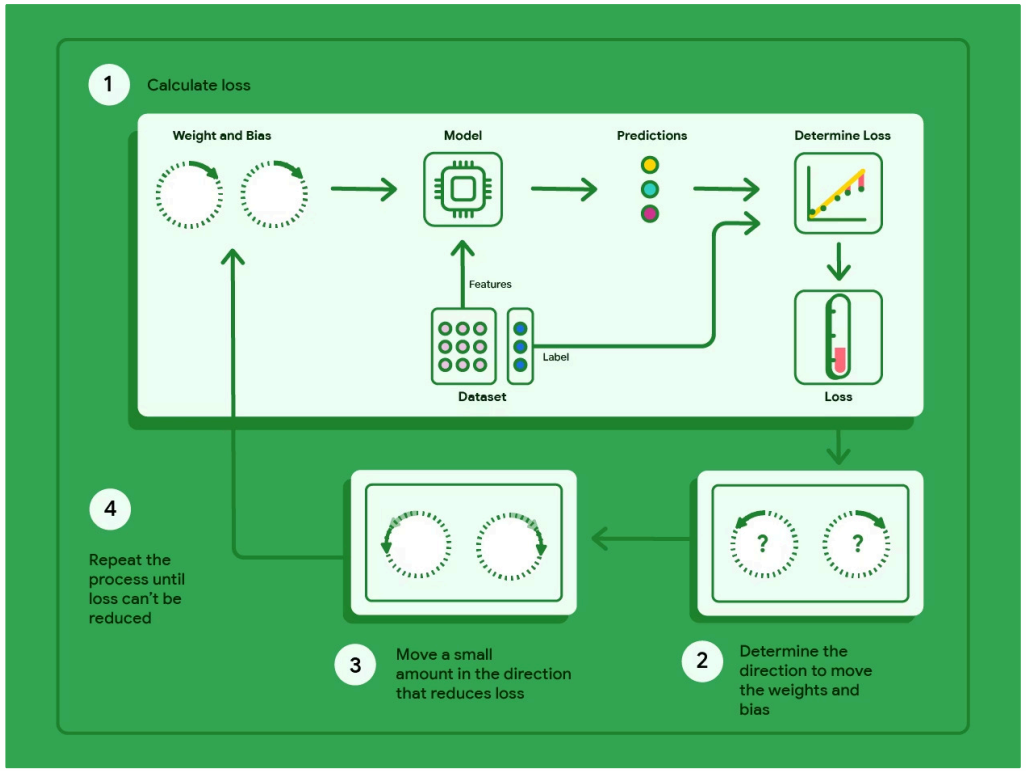


Analogía

Imaginá que estás en la cima de una montaña con niebla y querés llegar al valle (mínima loss). No podés ver el camino completo, pero podés sentir la pendiente bajo tus pies. En cada paso caminás un poco hacia abajo. Eso es gradient descent.

Learning rate = tamaño del paso: muy grande → te pasás, muy chico → tardás una eternidad.

Machine Learning Training Loop



01

Calculate Loss

Pass weight and bias through the model with dataset features and labels to generate predictions, then determine loss.

02

Determine Direction

Determine the direction to move the weights and bias in order to reduce loss.

03

Update Weights

Move a small amount in the direction that reduces loss.

04

Repeat

Repeat the process until loss can't be reduced further.

Backpropagation

En una red con muchas capas, ¿cómo sabemos cuánto ajustar cada peso?

Intuición: en el fondo una red neuronal es una función "diferenciable" (aproximadamente)

01

Forward pass

El input viaja hacia adelante por la red hasta producir una predicción

02

Cálculo de loss

Se mide el error entre predicción y valor real

03

Backward pass

El error se propaga hacia atrás, calculando cuánto contribuyó cada peso

04

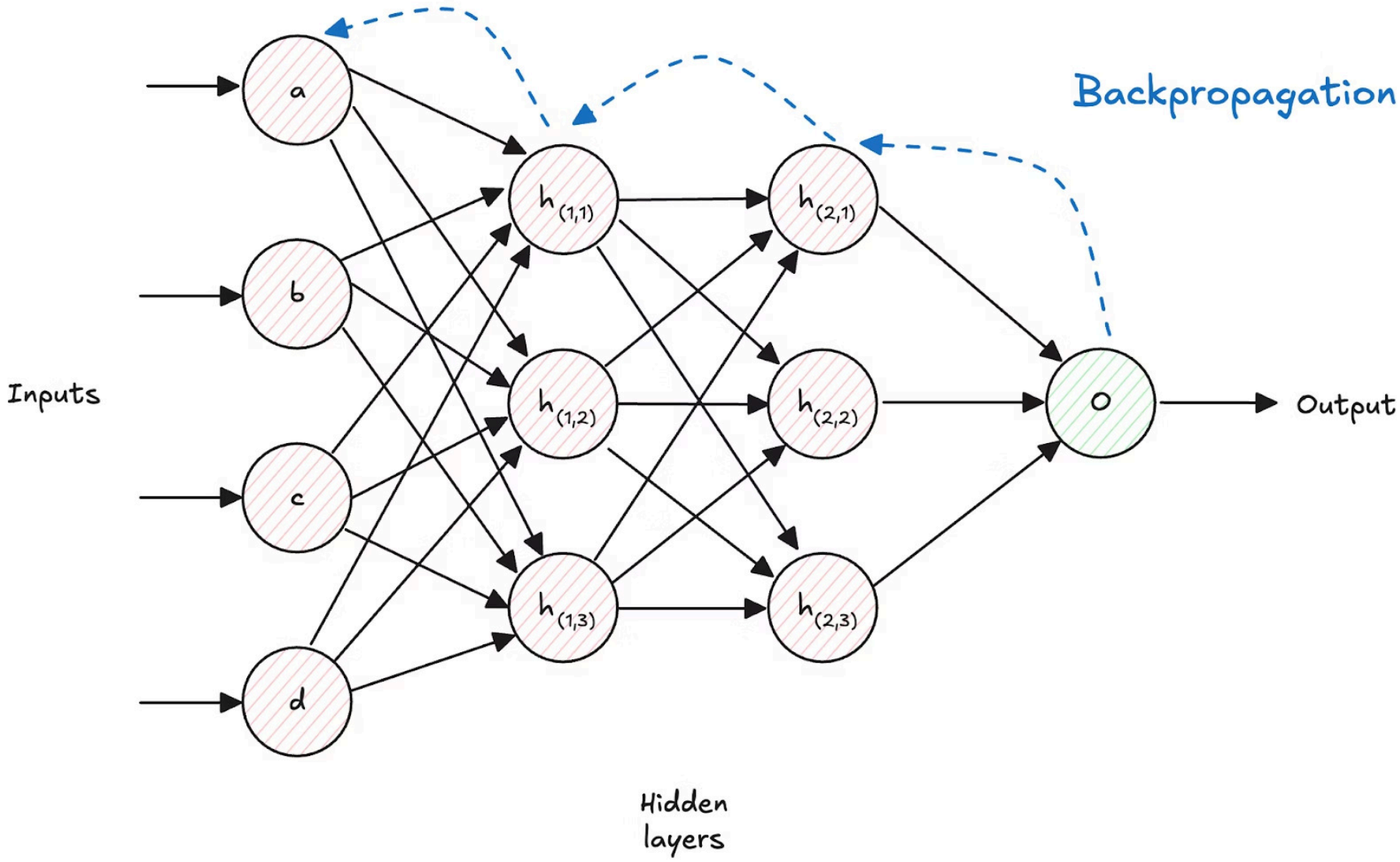
Actualización

Cada peso se ajusta proporcionalmente a su contribución al error

Es una forma de aplicar la regla de la cadena (sacar el gradiente de una composición de funciones), en una red neuronal.

Es como rastrear un error en una cadena de producción: el producto final salió mal, entonces vamos viendo en cada paso quién tuvo más culpa y corregimos más a ese.

Esto es lo que permite entrenar redes con millones de parámetros.


Backpropagation
Inputs
a
b
c
d
h(1,1)
h(1,2)
h(1,3)
h(2,1)
h(2,2)
h(2,3)
O
Output
Hidden layers

Redes Recurrentes (RNNs)

Antes de Transformers, las RNNs procesaban texto palabra por palabra, de izquierda a derecha, manteniendo un estado interno ("memoria").

```
"El" → actualiza estado → "gato" → actualiza estado → "está" → ...
```

Variantes: LSTM (decide qué recordar/olvidar), GRU (simplificada), ELMo (embeddings contextuales).

Limitaciones

- Procesamiento secuencial → no se puede paralelizar → lento
- Dificultad con dependencias largas: la información se "diluye" con la distancia

El problema del contexto

```
"The animal didn't cross the street because it was too tired"
```

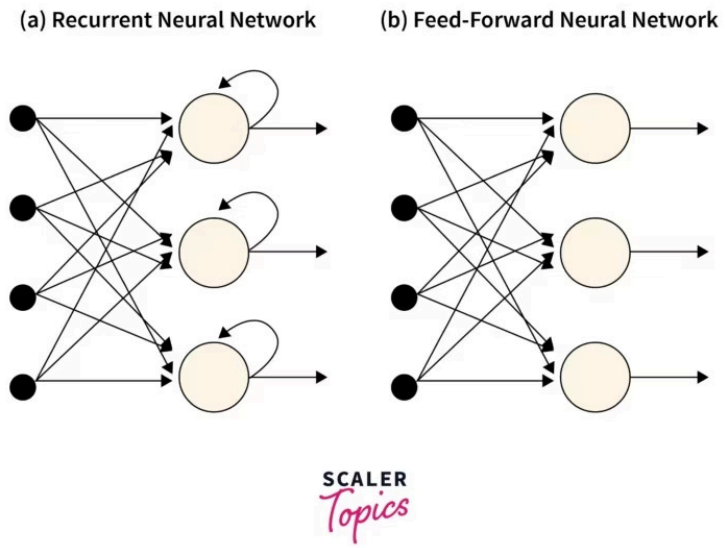
¿A qué se refiere "it"? A "animal". Pero están lejos en la oración.
Para cuando la RNN llega a "it", la info sobre "animal" se debilitó → motivó los Transformers.

Redes Recurrentes (RNNs): intuición

Quiero procesar: "The cat is on the _"

Red neuronal común: recibe el vector de "The" y genera un output, luego recibe el vector de "cat" y genera un output. Pero el output de "cat" va a ser siempre el mismo! Porque no tiene "memoria".

RNN: para procesar el vector de "cat", guardate el vector de "The" y usalo para generar la salida de "cat" (aplicar este mismo proceso para toda la frase hasta llegar a la ultima palabra).



Transformers

La arquitectura que cambió todo (2017). A diferencia de las RNNs, los Transformers procesan toda la secuencia de una vez, en paralelo.

Intuición de la Atención

Cuando leemos, no prestamos la misma atención a todas las palabras:

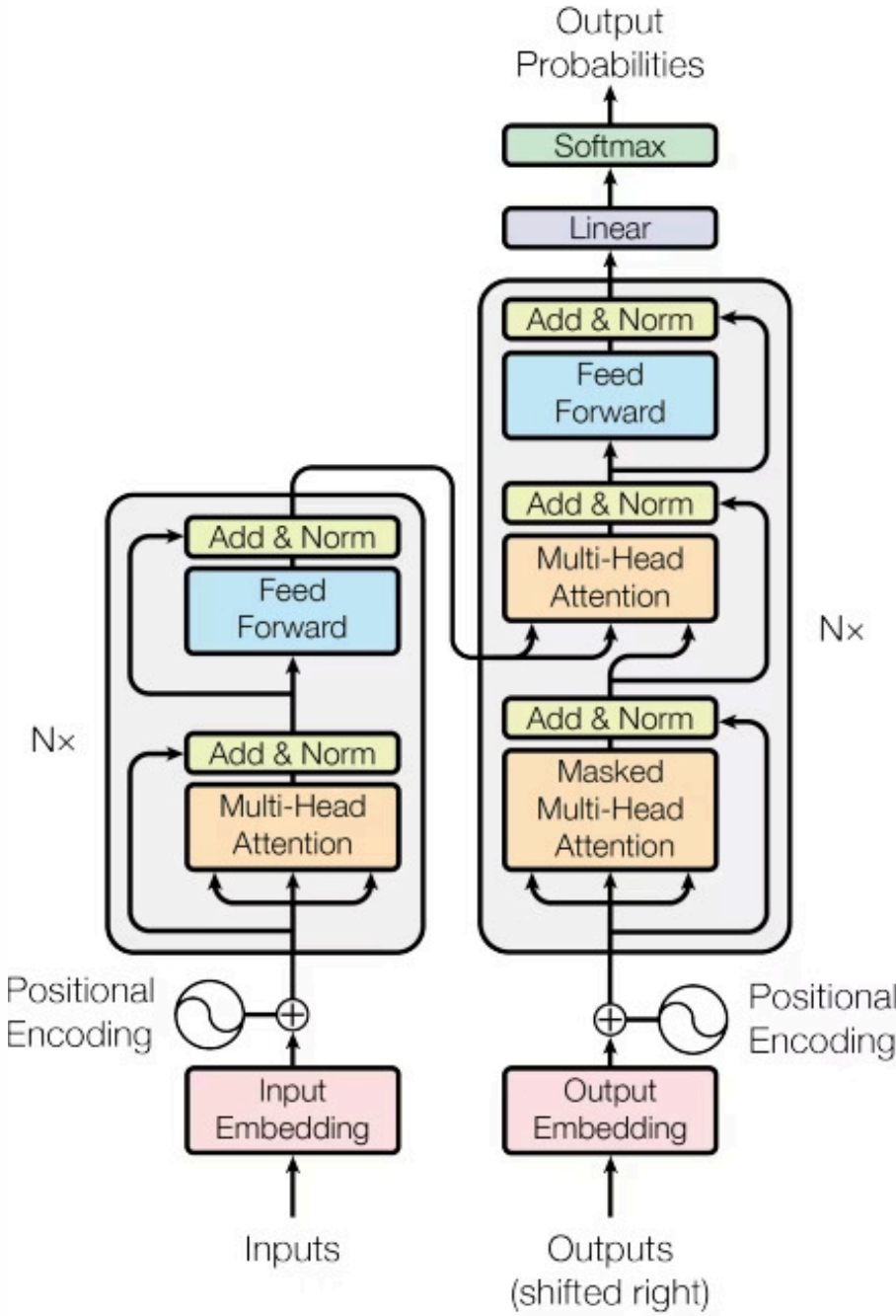
> "The animal didn't cross the street because it was too tired"
> → Para entender "it", miramos "animal". Eso es atención.

Atención permite que cada token "mire" a todos los demás y decida cuáles son relevantes.

- Captura dependencias largas (palabras separadas que se relacionan)
- Relaciones sintácticas (sujeto-verbo-objeto)
- Relaciones semánticas (significado, contexto)

Arquitectura

Paper original: encoder + decoder. Pero LLMs modernos (GPT, LLaMA, Claude) usan solo decoder.



Comparación de Modelos

Modelo	Idea principal	Novedad	Pros	Contras
Word2Vec	Aprende embeddings a partir del contexto en que aparecen las palabras	Demostró que se pueden aprender representaciones semánticas de forma no supervisada. Palabras similares quedan cerca en el espacio vectorial.	Simple, rápido de entrenar. Captura analogías semánticas (rey - hombre + mujer ≈ reina).	Cada palabra tiene un solo vector sin importar el contexto. No modela el orden de las palabras ni secuencias.
RNNs (LSTM, GRU)	Procesan texto secuencialmente manteniendo un estado interno que actúa como memoria	Introdujeron la capacidad de modelar secuencias y dependencias temporales en texto. Permitieron tareas como traducción y generación.	Capturan el orden de las palabras. LSTM/GRU mejoran la memoria a mediano plazo. Base de modelos como ELMo.	Procesamiento secuencial (no paralelizable, lento). Pierden información en secuencias largas. Difícil capturar contexto distante.
Transformers	Usan attention para que cada token mire a todos los demás tokens en paralelo	Eliminaron la necesidad de procesar secuencialmente. Capturan dependencias largas sin degradación. Base de todos los LLMs modernos.	Altamente paralelizables (entrenan rápido en GPUs). Capturan contexto largo. Escalan muy bien con más datos y parámetros.	Requieren enormes cantidades de datos y cómputo. Alto costo de entrenamiento. El mecanismo de attention crece cuadráticamente con la longitud.

Evolución: representaciones estáticas → secuenciales → contextuales con attention.

Cómo funciona un LLM

Cuando le mandás un mensaje a ChatGPT/Claude, internamente pasa esto:

01

Tokenización

Tu texto se divide en tokens → "¿Cómo estás?" → ["¿","Cómo","est","ás","?"]

02

Embeddings

Cada token se convierte en un vector numérico

03

Transformer

Los vectores pasan por muchas capas de attention

04

Vector de probabilidades

La última capa produce un vector de tamaño |V|

05

Selección

Se elige un token (con algo de aleatoriedad controlada = "temperatura")

06

Repetición

Se agrega al texto y se repite desde el paso 1

> **Los modelos generan texto token por token.**
>
> Cada vez que ves una palabra aparecer en ChatGPT, es un ciclo completo de este proceso.

Generación de Palabras — Escala LLM

El mismo problema de Language Modelling, pero a una escala masiva:

	Ejemplo anterior	ChatGPT (GPT-4)
Vocabulario V	14 palabras ("a", "to", "cat", ...)	~100.000 tokens (sub-palabras en múltiples idiomas)
Corpus T	5 frases	Billones de palabras (internet, libros, código, papers...)
Parámetros	Pocos	Cientos de miles de millones
Entrenamiento	Segundos	Meses en miles de GPUs
Frase P	"the dog sat on"	Cualquier texto, de cualquier largo

El concepto es exactamente el mismo: predecir la siguiente palabra.
Lo que cambia es la escala del vocabulario, los datos y el modelo.

¿Qué cambió?

Muchas tareas se resuelven ahora con una sola técnica: generación de texto con **modelos fundacionales** (modelos grandes pre-entrenados).

Si un modelo puede comprender texto y generar palabras, podemos reformular casi cualquier tarea como generación:

- Sentimiento: "¿Este review es positivo o negativo?" → modelo genera "positivo"
- Traducción: "Traducí: the cat is on the mat" → modelo genera "el gato está sobre la alfombra"
- Resumen: "Resumí este texto: [texto largo]" → modelo genera el resumen

```
problema → prompt (instrucción en texto) → modelo genera la solución
```

Esto es lo que hacemos cuando usamos ChatGPT, Claude, etc.

Entrenamiento vs. Inferencia

	Entrenamiento	Inferencia
¿Qué pasa?	Aprende ajustando parámetros	Usa parámetros ya aprendidos
¿Cuándo?	Antes de estar disponible	Cada vez que un usuario pregunta
¿Quién?	OpenAI, Anthropic, Meta...	Vos, al usar ChatGPT o Claude
Ejemplo	Entrenar GPT-4	Preguntarle "¿qué es NLP?"
Costo	Millones de dólares, semanas	Fracción de segundo por respuesta
Parámetros	Se modifican constantemente	Están congelados (no cambian)

Cuando usás ChatGPT, el modelo no está aprendiendo de tu conversación. Los parámetros ya están fijos. Solo está haciendo inference.

Entrenar un modelo grande requiere billones de palabras y miles de GPUs durante semanas. Por eso muy pocas empresas lo hacen.